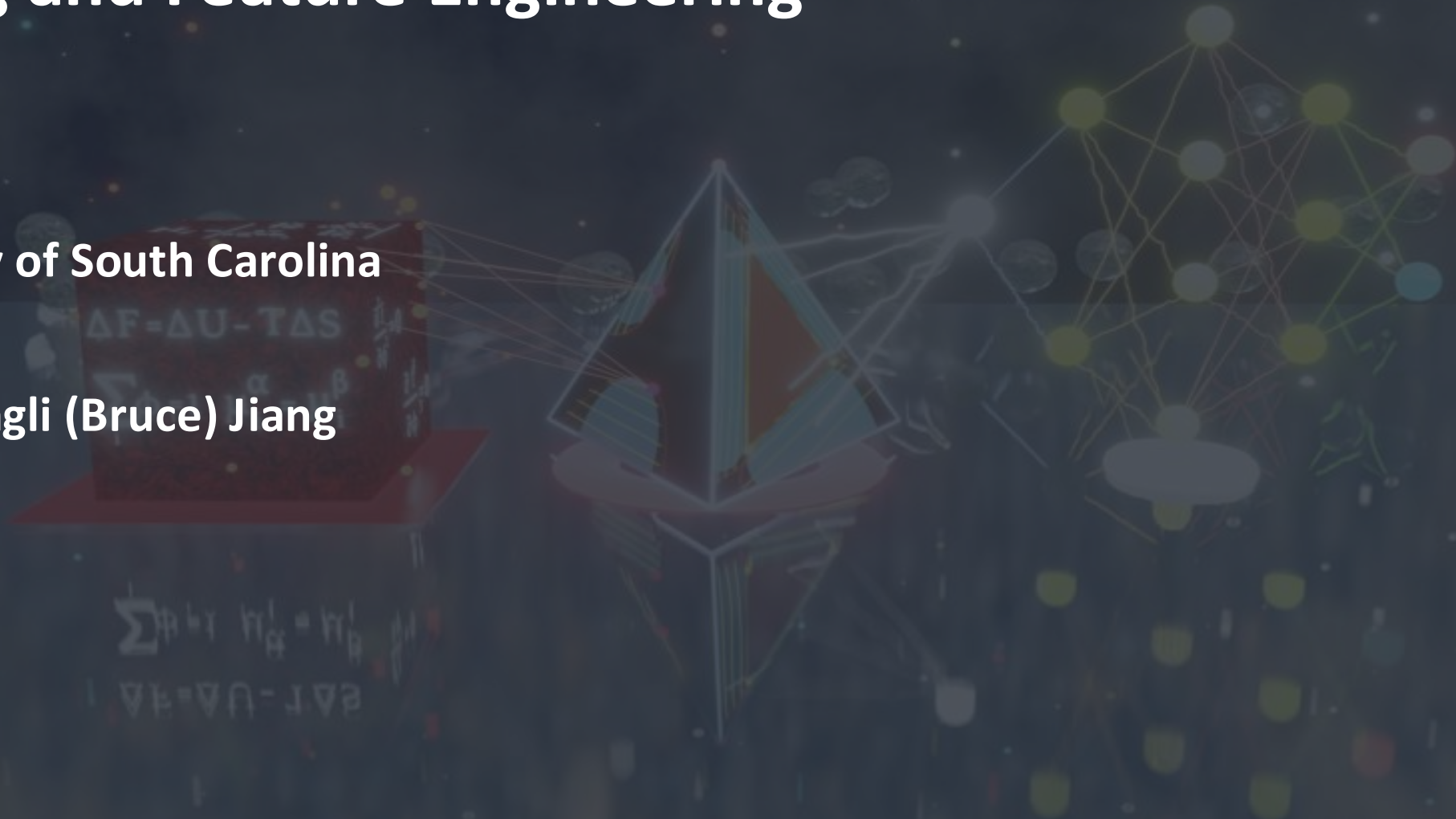


Scaling and Feature Engineering

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



From Lecture 9

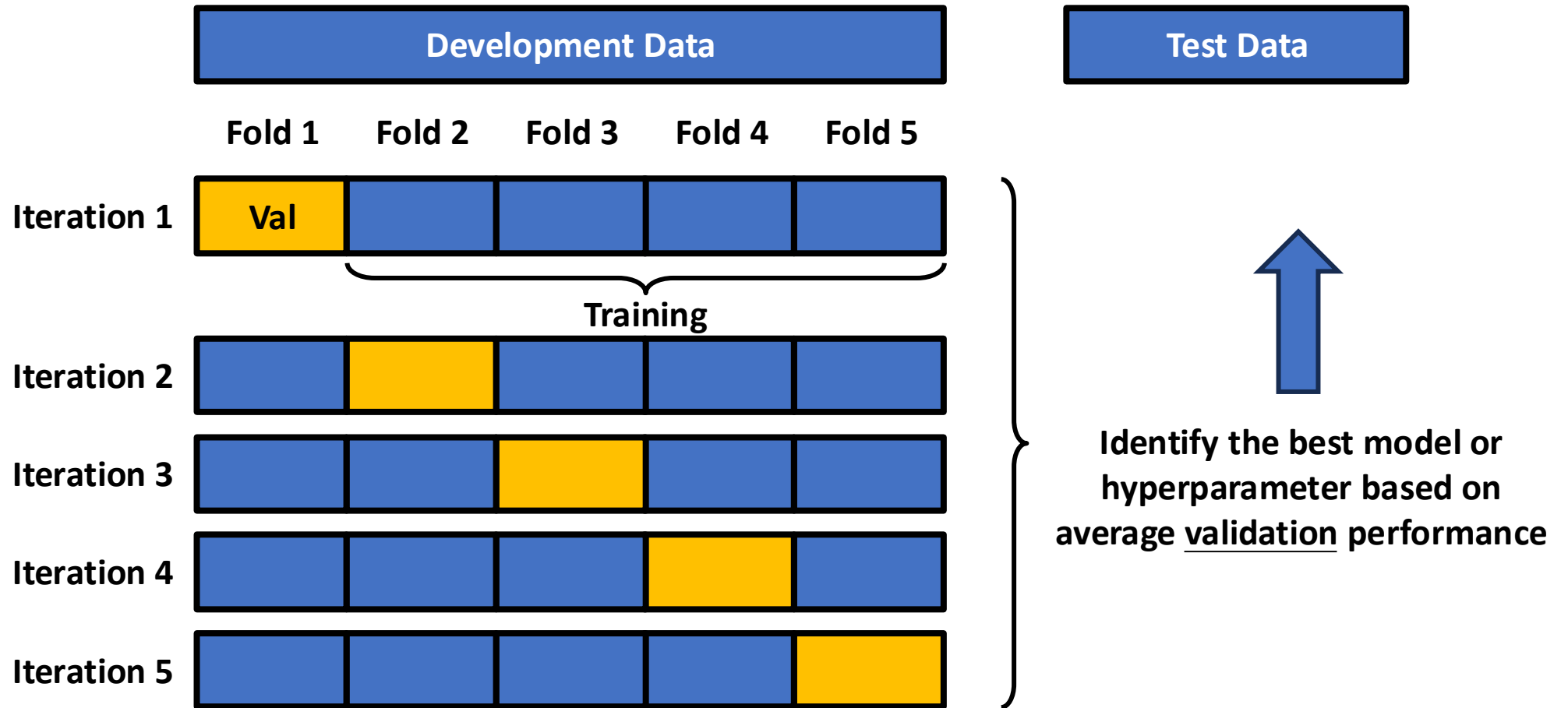
Model Selection & Hyperparameter Tuning

- *We aim to develop accurate models that **generalize** to unseen data.*
- *We seek the appropriate balance between **model complexity** and the **amount of available data**.*
- ***Validation data** helps us select hyperparameters and estimate generalization performance.*

In this lecture, you will learn

- *How to scale and transform input features and target values.*
- *How to construct polynomial features and interaction terms.*

Cross-Validation Revisit



Why use k-fold cross-validation? A single validation split may be unusually easy or difficult. Testing several splits and averaging their scores gives a more reliable estimate of performance on unseen data.

K-Fold CV vs. a Single Validation Split

Situation	A sensible choice
Reproducing a paper or official benchmark	Follow its prescribed split and evaluation procedure.
Small or moderate dataset; training is affordable	Start with 5-fold CV on the development data. This reduces dependence on one particular validation group.
Large, representative validation set; training is expensive	A single training–validation–test split is often sufficient.

The original paper used a 41/43/40 training–validation–test split.
How should we evaluate our model for a fair comparison?

The Same Regression Problem

$$\Delta Q(V) = Q_{100}(V) - Q_{10}(V)$$

Where $Q_n(V)$ is the discharge capacity at voltage V during cycle n . Each curve contains 1000 voltage points.

Targets	Meaning
cycle_life	Total battery lifetime, in cycles, using the released 80%-capacity lifetime label. It is not the number of cycles remaining after cycle 100.

Features

Variance of ΔQ

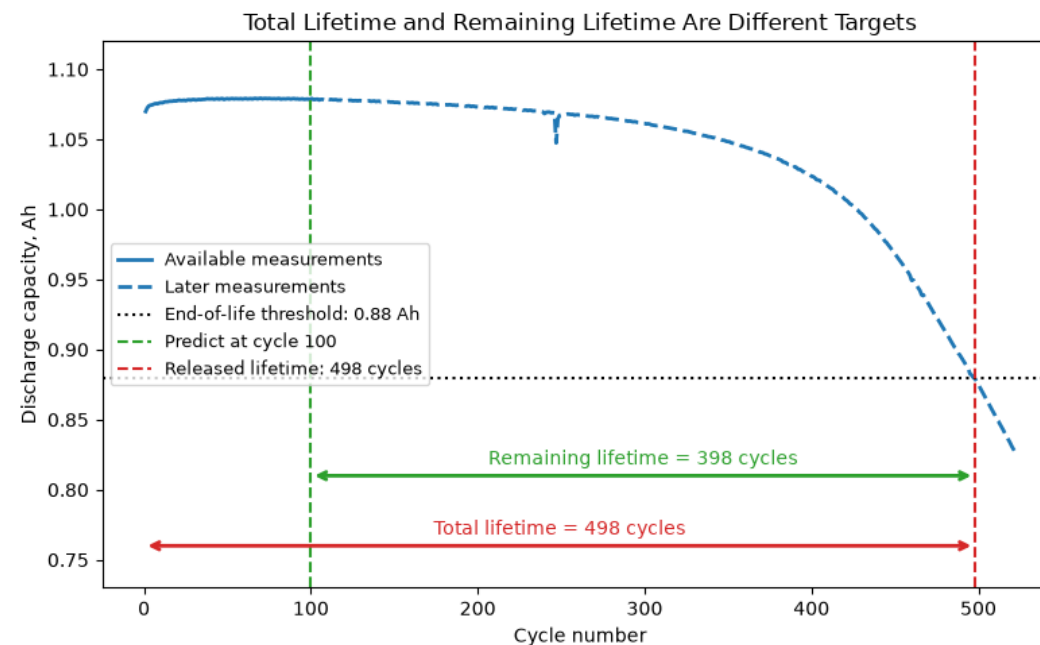
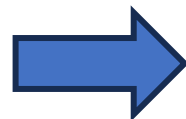
Magnitude of minimum ΔQ

Mean ΔQ

Maximum capacity at cycle 2

Change in maximum capacity

Fitted maximum-capacity slope



Can we beat our **125-cycle** val RMSE and **193-cycle** test RMSE?
Paper reference: **91-cycle** val RMSE and **173-cycle** test RMSE.

Improving Data Representation

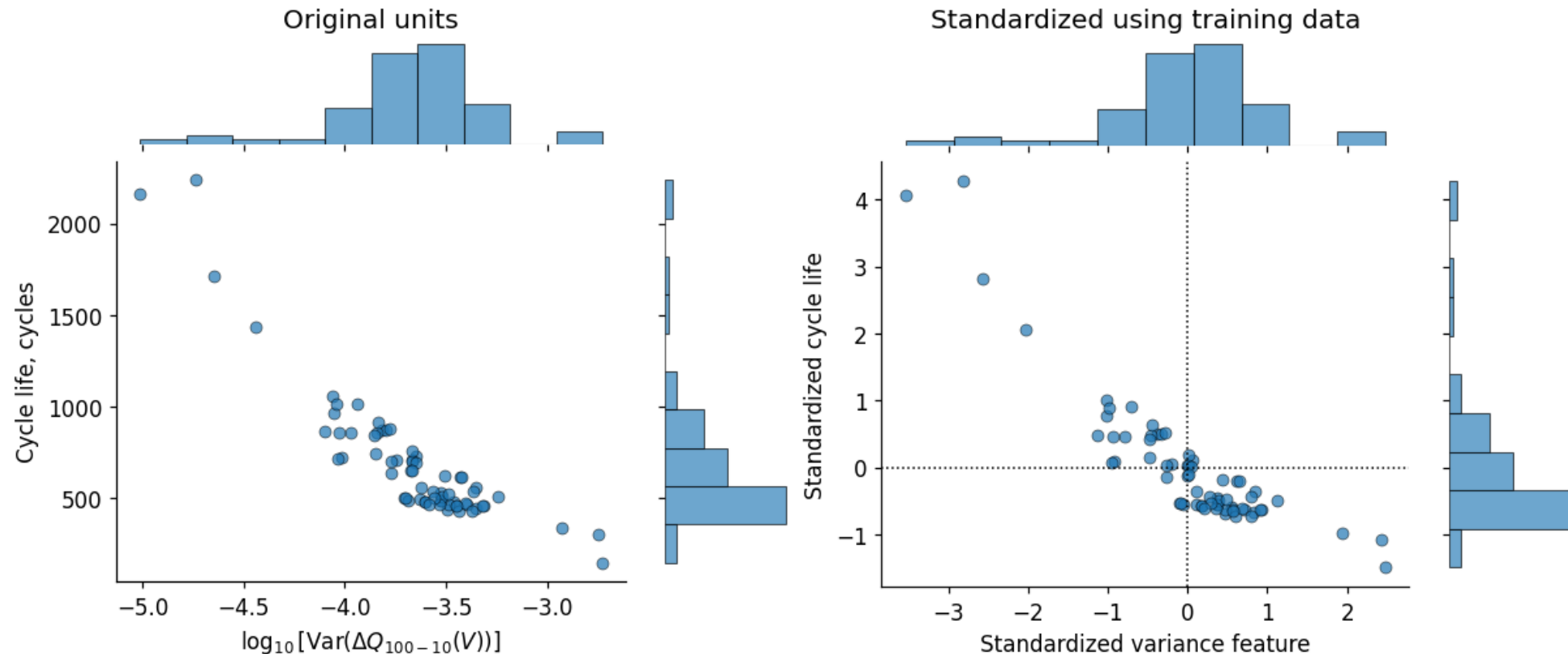
Method	Definition	Examples
Linear Scaling	Change the range or units of existing variables	Standard, min-max
Nonlinear Transformation	Changes the spacing or distribution of values	Log, Yeo-Johnson
Feature engineering	Creates new features from existing measurements	Ratios, squares, interactions

All of these choices belong to the **model-selection** procedure.

Linear Scaling: StandardScaler

Definition: Subtract the training mean and divide by the training standard deviation.

`from sklearn.preprocessing import StandardScaler`

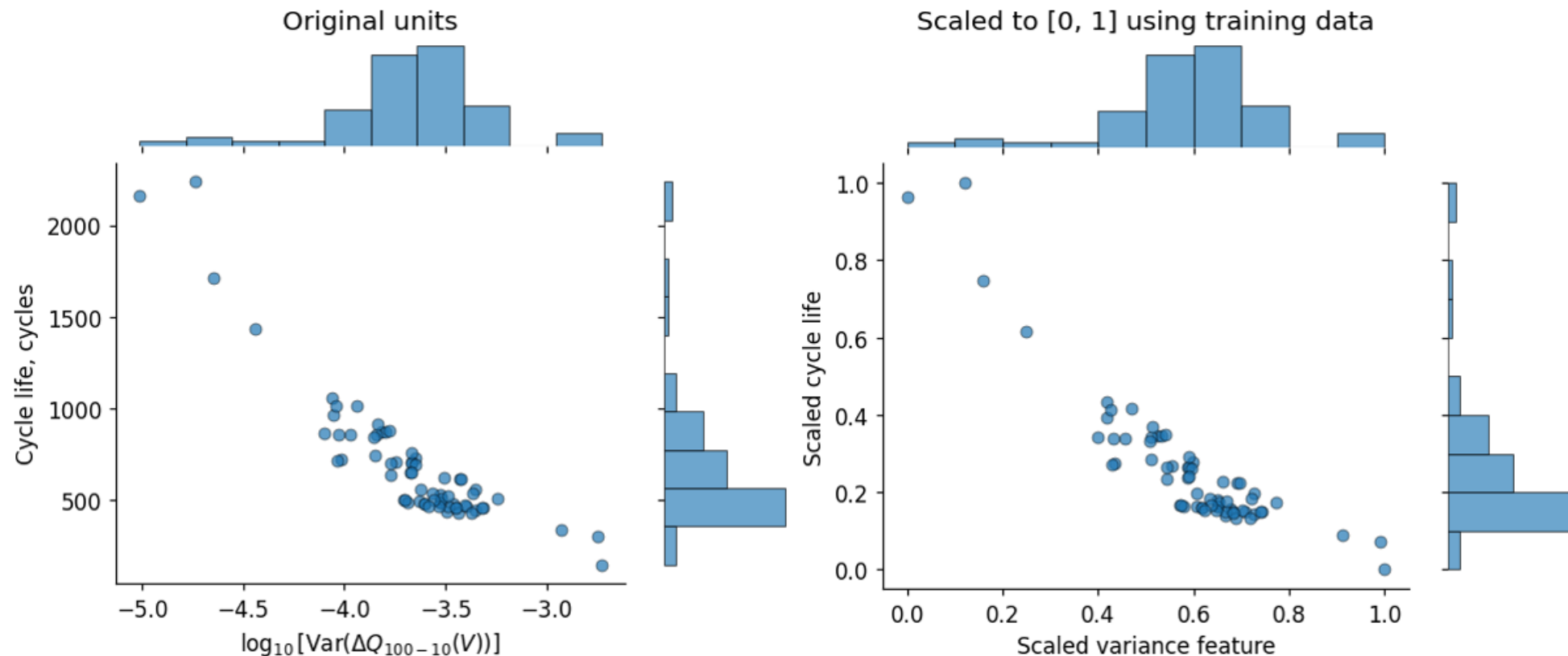


**Training mean = 0 and training standard deviation = 1.
Distribution shape and correlation are preserved.**

Linear Scaling: MinMaxScaler

Definition: Subtract the training minimum and divide by the training range.

`from sklearn.preprocessing import MinMaxScaler`



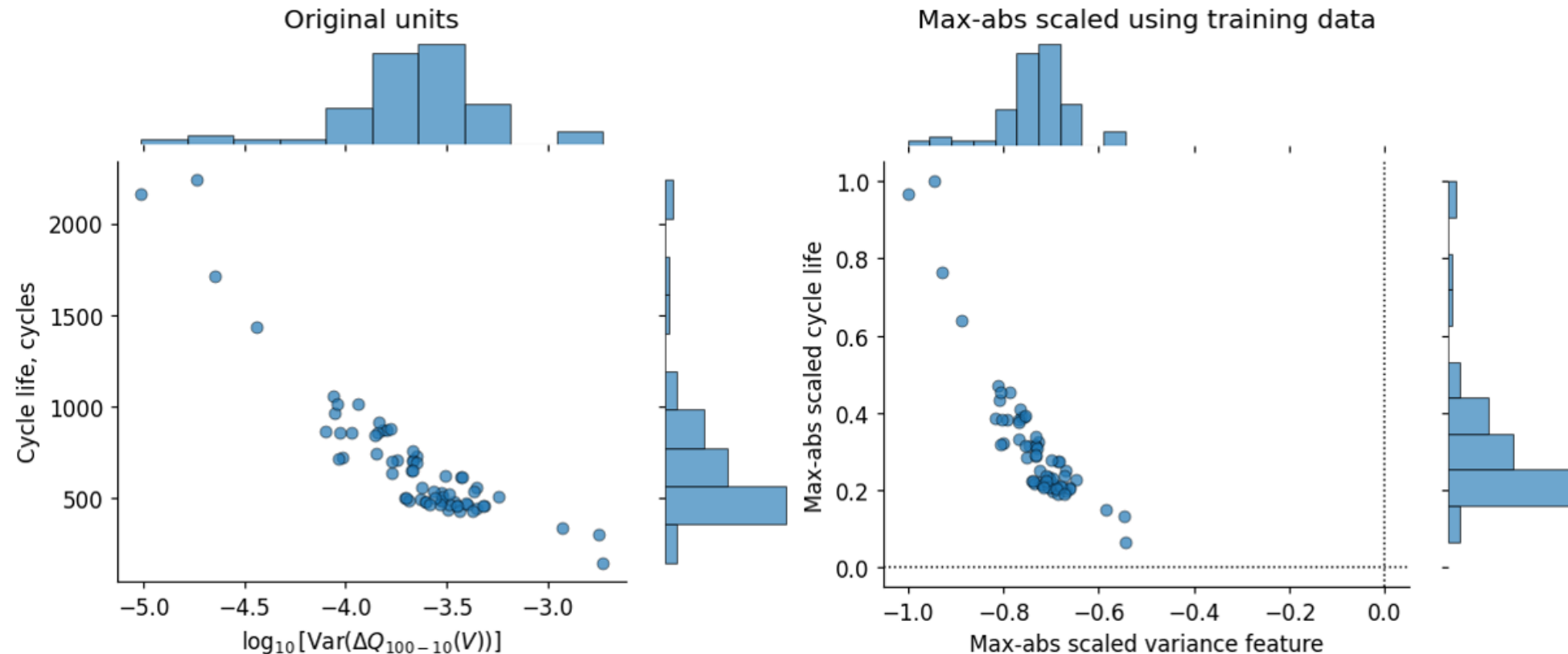
$$x_{\text{scaled}} = \frac{x - x_{\min, \text{train}}}{x_{\max, \text{train}} - x_{\min, \text{train}}}$$

Training min = 0 and training max = 1. Distribution shape and correlation are preserved. **Outliers** can compress the remaining values.

Linear Scaling: MaxAbsScaler

Definition: Divide each value by the largest absolute value in the training data.

`from sklearn.preprocessing import MaxAbsScaler`



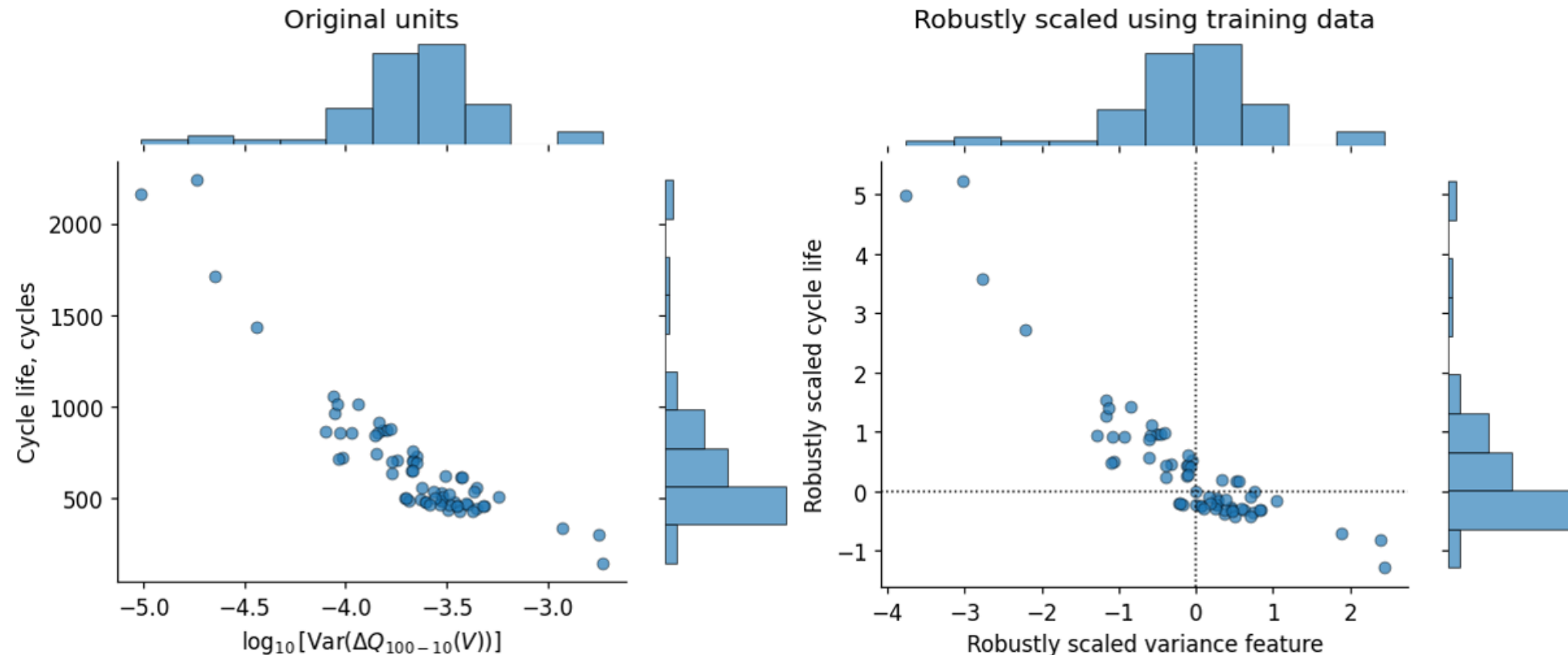
$$x_{\text{scaled}} = \frac{x}{\max_{i \in \text{train}} |x_i|}$$

Largest training absolute value = 1. Zero and signs are preserved.

Linear Scaling: RobustScaler

Definition: Subtract the training median and divide by the training interquartile range (IQR).

`from sklearn.preprocessing import RobustScaler`



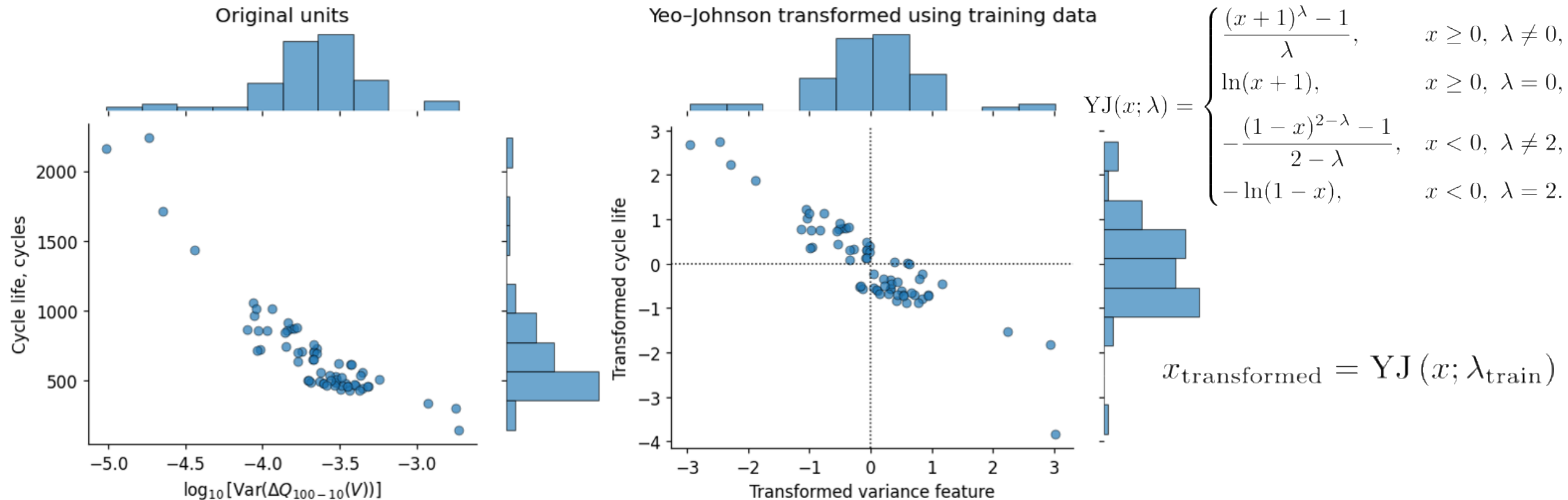
$$x_{\text{scaled}} = \frac{x - \text{median}(x_{\text{train}})}{Q_{0.75}(x_{\text{train}}) - Q_{0.25}(x_{\text{train}})}$$

Training median = 0 and training IQR = 1.
Outliers have less influence on the scaling, but they are not removed.

Nonlinear Transformation: Yeo–Johnson

Definition: Learn a power transformation from the training data to make data more **Gaussian-like**.

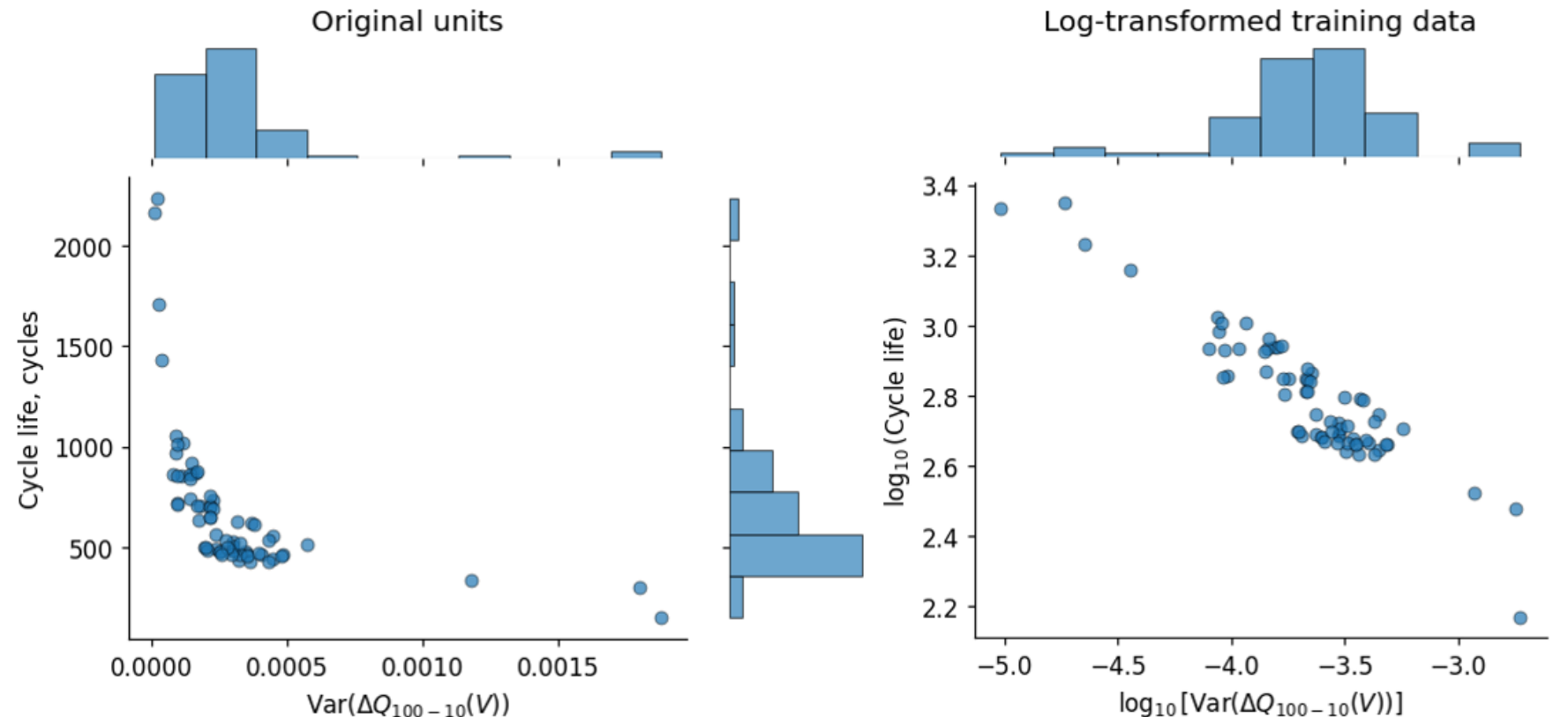
`from sklearn.preprocessing import power_transform`



λ is learned from the training data. The transformation can reduce skewness.
Order is preserved, but **distances and correlation may change**.

Nonlinear Transformation: Log

Definition: Apply a logarithm to positive values to compress large values and reduce skew.



$$x_{\text{transformed}} = \log_{10}(x), \quad x > 0$$

Log transformation requires positive values and changes distances between observations. What if we have negative values?

Scaling/Transformation Families

Method	What it does	Main caution
None	Keeps the original units	Larger-magnitude features may dominate
StandardScaler	Sets training mean to 0 and SD to 1	Sensitive to outliers
MinMaxScaler	Maps the training minimum and maximum to 0 and 1	Extremes compress the remaining values
MaxAbsScaler	Divides by the largest training absolute value	Does not center and is sensitive to extremes
RobustScaler	Sets training median to 0 and IQR to 1	Does not reduce skewness
Yeo–Johnson	Learns a power transformation to reduce skewness	Changes spacing and may change correlation
Log	Compresses large positive values	Requires positive values

Before evaluating target performance, use `target_transformer.inverse_transform(...)` to return predictions to the original units.

Which Method Would You Choose?

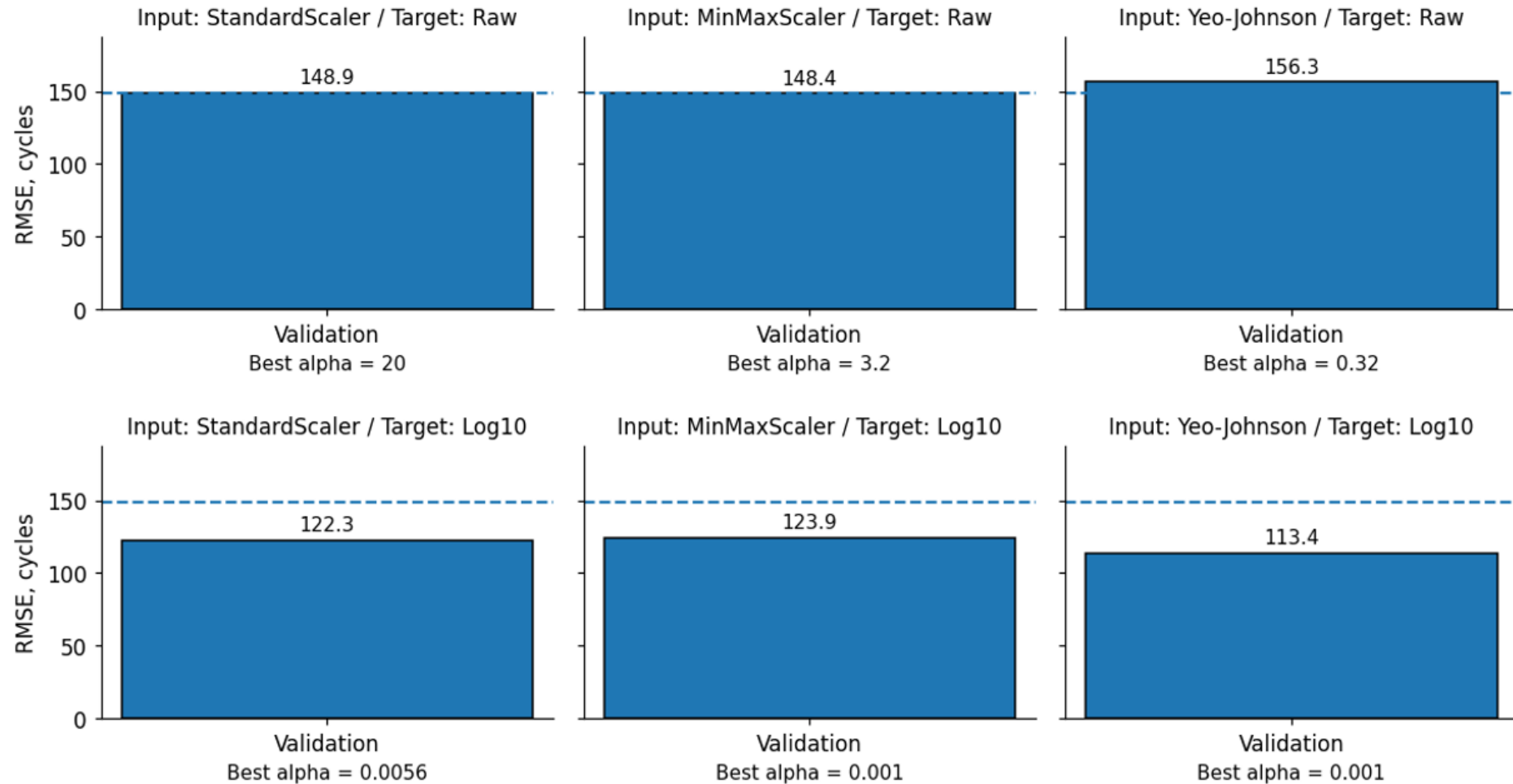
Choose from: StandardScaler, MinMaxScaler, MaxAbsScaler, RobustScaler, Yeo–Johnson, or Log.

1. Features have different units and no strong outliers.
2. Several extreme observations are present.
3. A positive variable has a long right tail.
4. A skewed variable includes zero and negative values.
5. The training range should map to 0 - 1.
6. Sparse features contain meaningful zeros.

Can transformations be combined?

Which Model Should We Select?

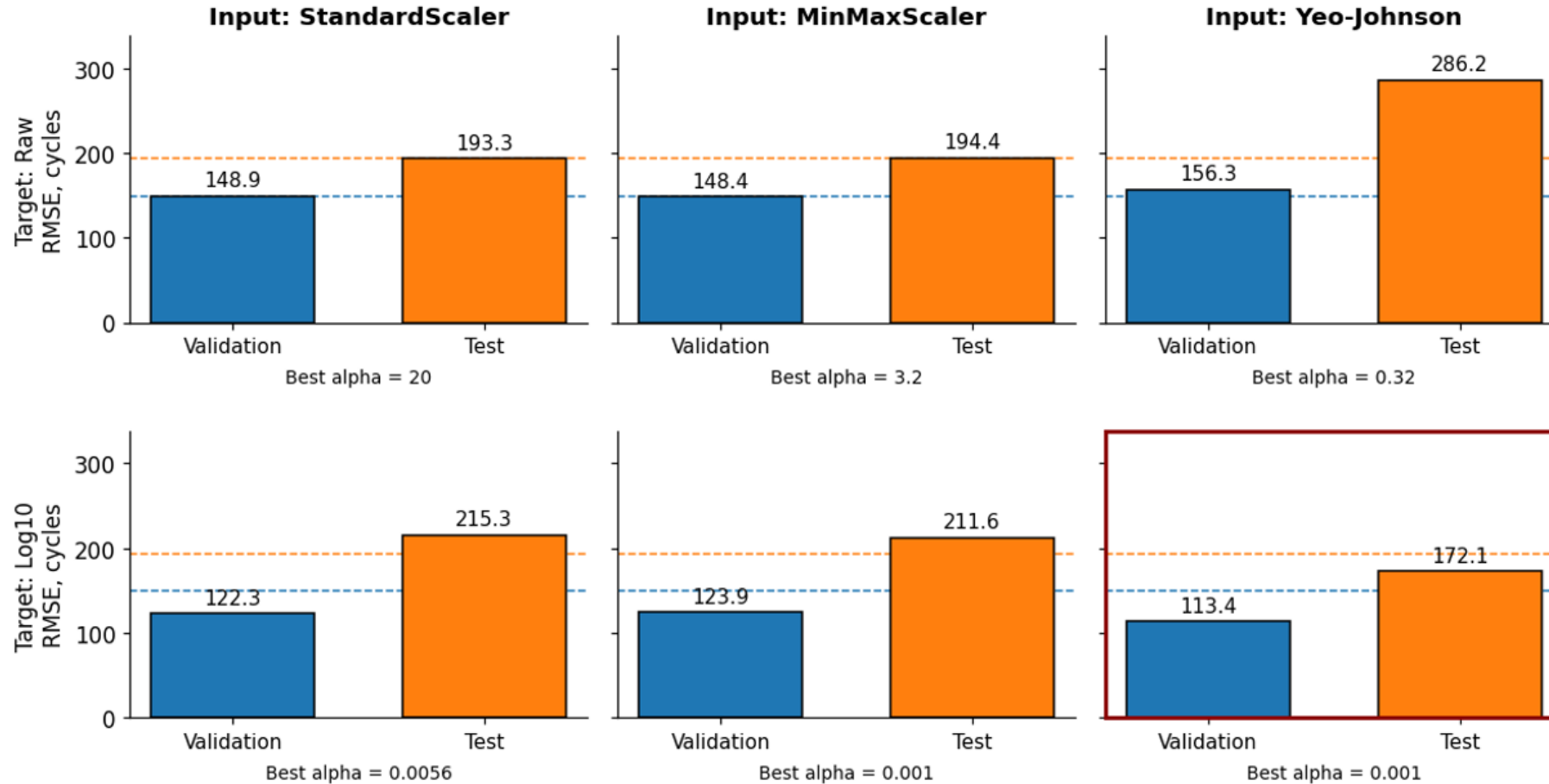
Lasso: input and target transformations
Fixed split, six stored features



Select models using validation performance. Use the test set only once for final evaluation.

Which Model Should We Select?

Lasso: input and target transformations
Fixed split; red outline = lowest validation RMSE



Dashed lines: StandardScaler inputs and raw target (validation 148.9, test 193.3 cycles)

Select models using validation performance. Use the test set only once for final evaluation.

Feature Engineering: Interaction Features

Definition: Add pairwise products while omitting squared terms.

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
```

```
X_train_poly = poly.fit_transform(X_train)
```

$$x_1, x_2 \longrightarrow x_1, x_2, x_1x_2$$

$$\hat{y} = \theta_0 + \theta_1x_1 + \theta_2x_2 + \theta_{12}x_1x_2$$

The effect of one input can change with the value of another input.

Feature Engineering: Polynomial Features

Definition: Create new features from powers and products of the original inputs.

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly = PolynomialFeatures(degree=2, include_bias=False)
```

```
X_train_poly = poly.fit_transform(X_train)
```

$$x_1, x_2 \longrightarrow x_1, x_2, x_1^2, x_1x_2, x_2^2$$

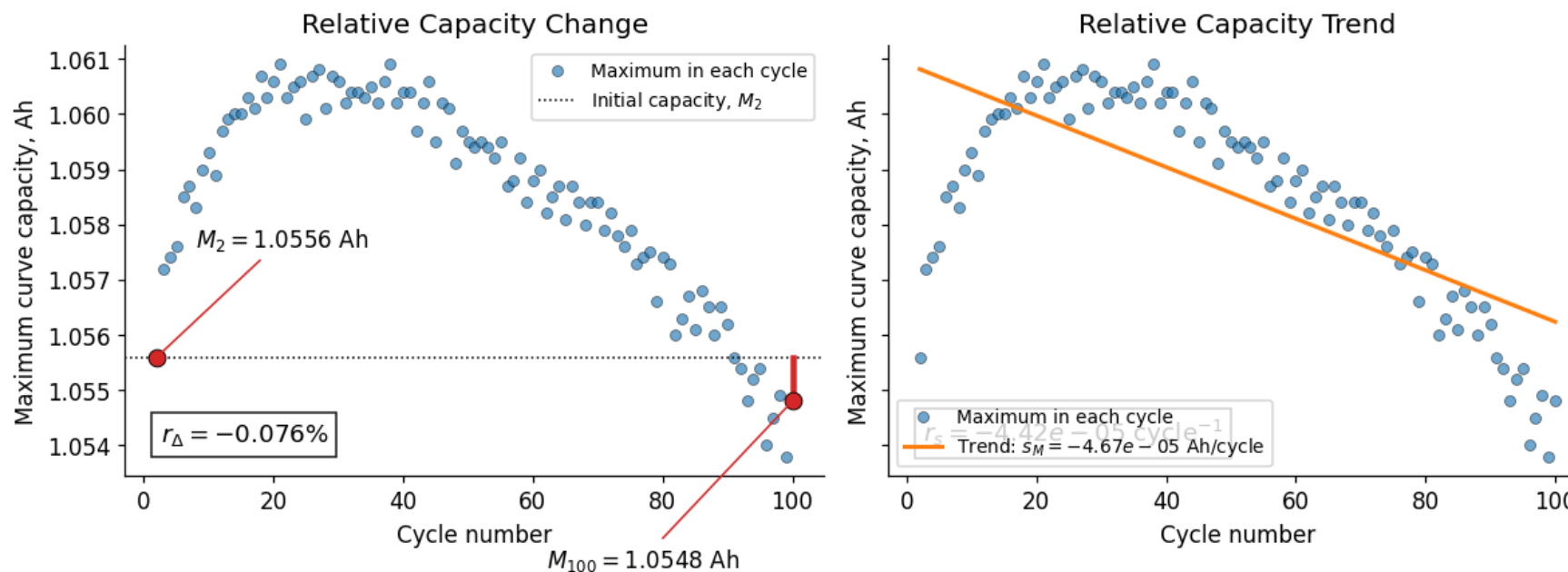
$$\hat{y} = \theta_0 + \theta_1x_1 + \theta_2x_2 + \theta_{11}x_1^2 + \theta_{12}x_1x_2 + \theta_{22}x_2^2$$

Squared terms allow curvature. Product terms allow interactions.

Feature Engineering: Polynomial Features

Definition: Create new features from powers and products of the original features.

Relative capacity change $r_{\Delta} = \frac{M_{100} - M_2}{M_2}$, $r_s = \frac{s_M}{M_2}$ Normalized capacity slope

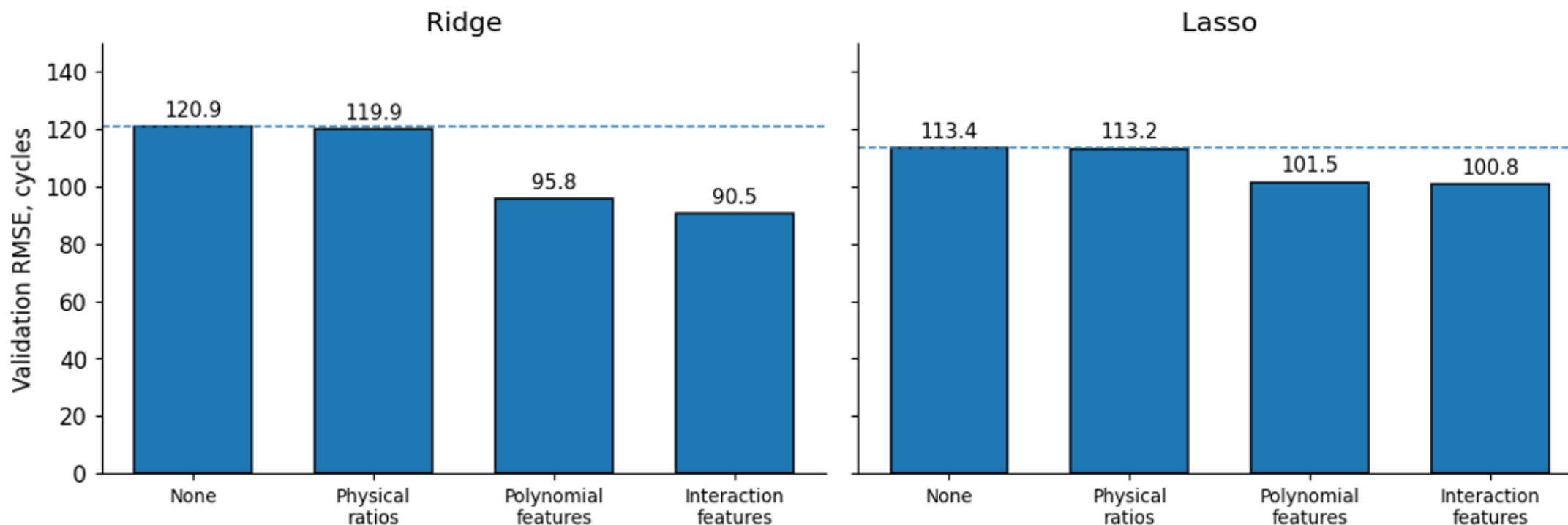


$$\hat{y} = \theta_0 + \sum_{j=1}^6 \theta_j x_j + \theta_7 r_{\Delta} + \theta_8 r_s$$

Example: Fixed Preprocessing

Yeo-Johnson inputs and a log10 target for every representation. Within each model, only the representation and its retuned alpha change.

Feature engineering with fixed preprocessing
Yeo-Johnson inputs, log10 target



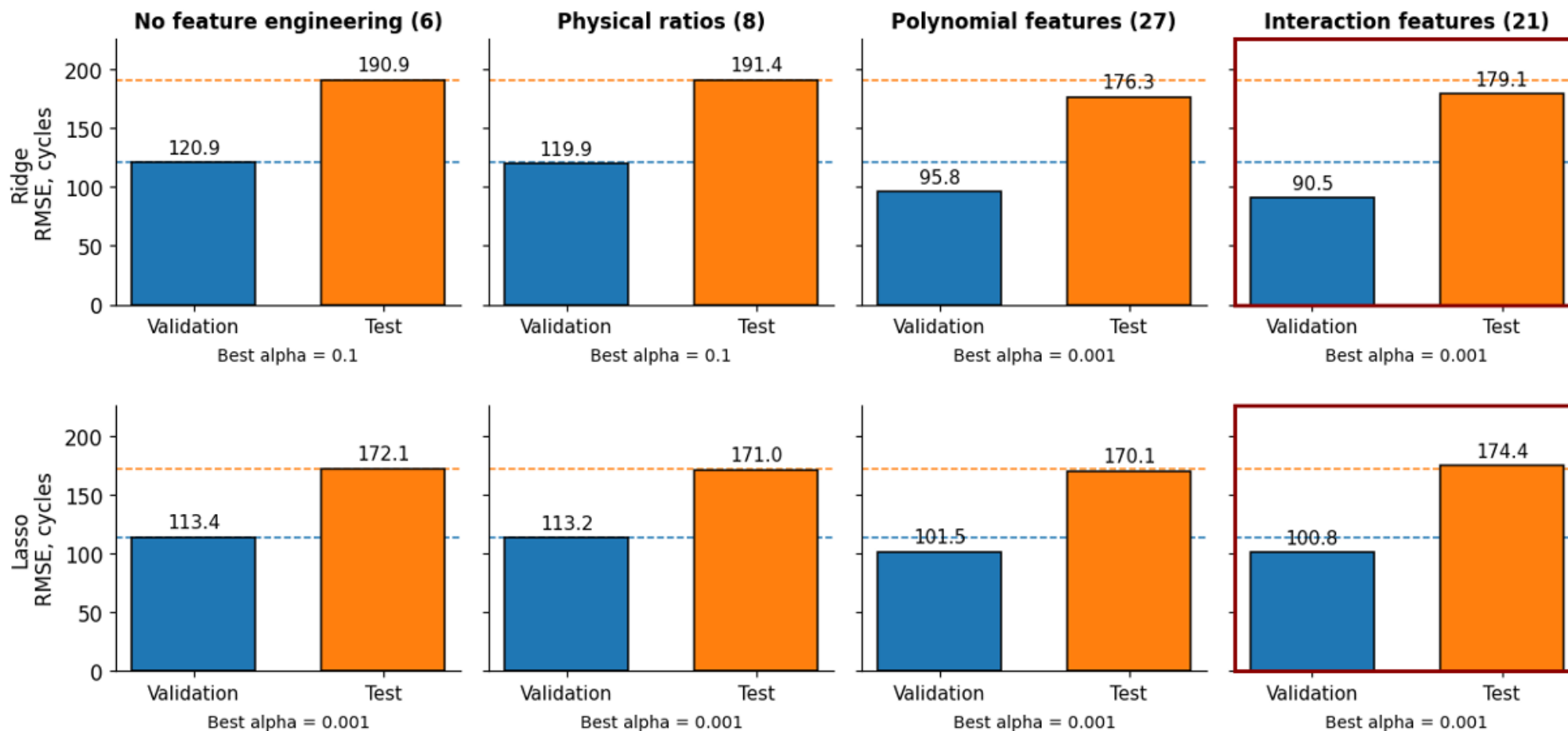
Why model should I pick?

Can we beat our **125-cycle** val RMSE and **193-cycle** test RMSE?
Paper reference: **91-cycle** val RMSE and **173-cycle** test RMSE.

Example: Fixed Preprocessing

When reporting model performance, should you include both validation and test results?

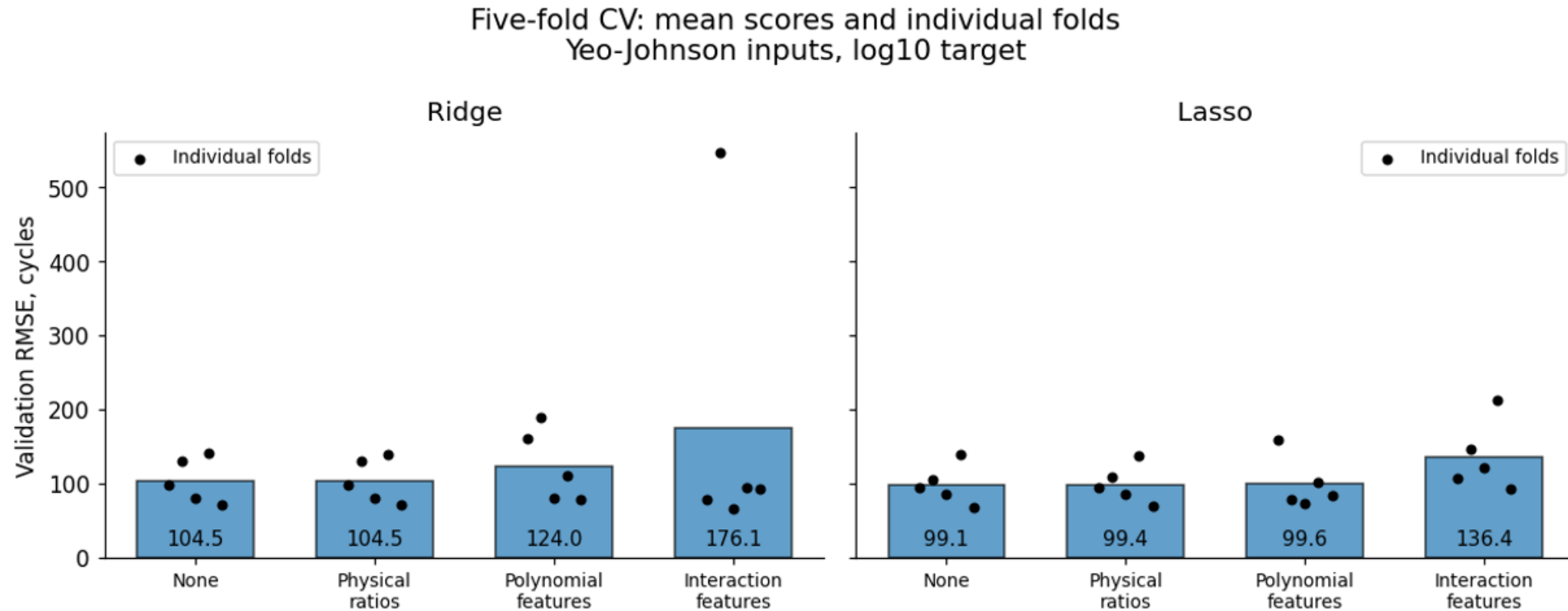
Feature engineering with fixed preprocessing
Yeo-Johnson inputs, log10 target; red = validation selection within each model



Report both. Validation results show how the model was selected and tuned; test results provide an unbiased estimate of performance on unseen data.

Example: Fixed Preprocessing

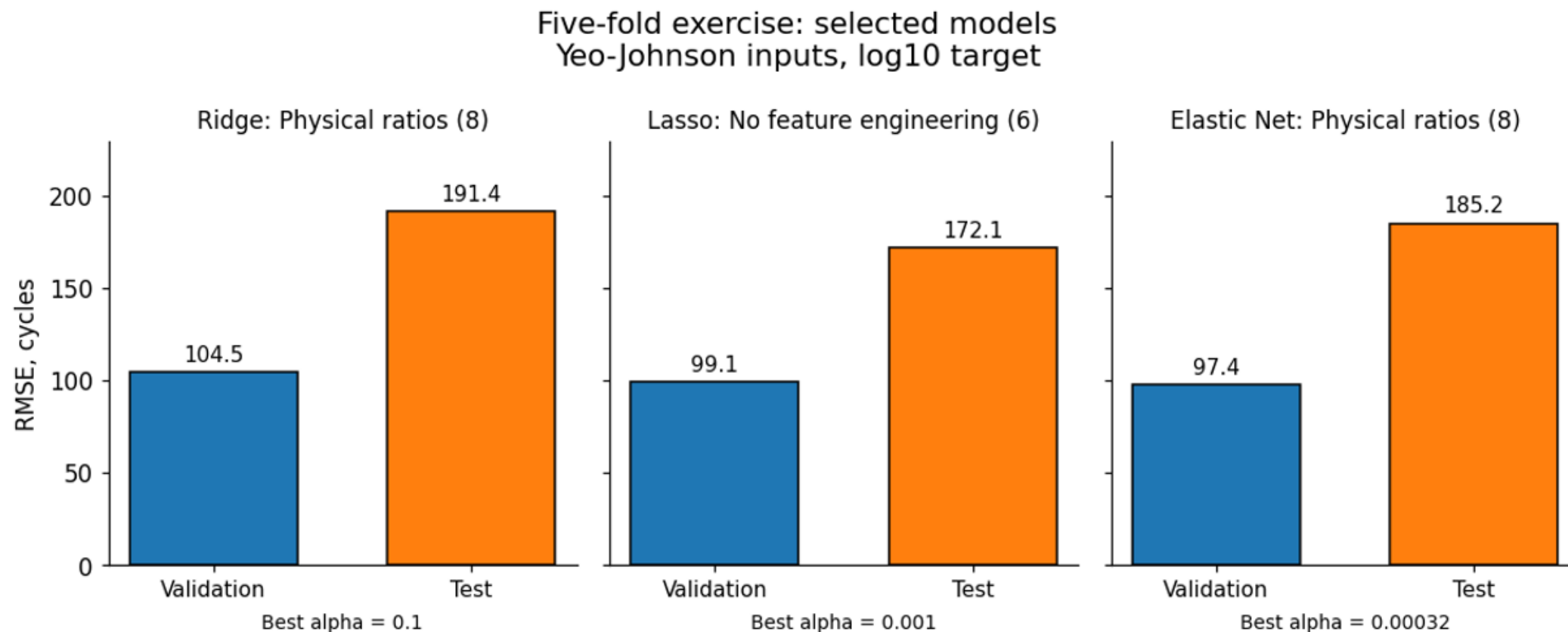
Yeo-Johnson inputs and a log10 target for every representation. Within each model, only the representation and its retuned alpha change.



Why model should I pick?

Example: Fixed Preprocessing

Yeo-Johnson inputs and a log10 target for every representation. Within each model, only the representation and its retuned alpha change.



Quiz

1. Which transformation changes the histogram shape? Which only changes the axis units?
2. Why must we return log-target predictions to cycles before comparing RMSE?
3. Why can the fixed-split and five-fold searches choose different hyperparameters?
4. The overall search yields the lowest validation RMSE, but another model has a lower test RMSE. Can you cherry-pick the model with the lowest test RMSE? Why or why not?

Quiz

- 1. A log and Yeo-Johnson transformation changes the histogram's shape; a linear scalar only changes the axis units.**
- 2. RMSE must compare predictions and targets in the same unit (cycles), not log-cycles.**
- 3. A fixed split depends on one partition; five-fold CV averages across multiple partitions, so rankings can differ.**
- 4. No. The test set is reserved for final, unbiased evaluation. Choosing a model based on test performance leaks test information into model selection and can produce an overly optimistic estimate. Select the model using validation or cross-validation RMSE, then evaluate it once on the test set.**

Validation and Test Performance in Literature

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
<i>SIFT + FVs [7]</i>	—	—	26.2%
1 CNN	40.7%	18.2%	—
5 CNNs	38.1%	16.4%	16.4%
1 CNN*	39.0%	16.6%	—
7 CNNs*	36.7%	15.4%	15.3%

Table 2: Comparison of error rates on ILSVRC-2012 validation and test sets. In *italics* are best results achieved by others. Models with an asterisk* were “pre-trained” to classify the entire ImageNet 2011 Fall release. See Section 6 for details.

ANI-1: an extensible neural network potential with DFT accuracy at force field computational cost†

J. S. Smith,^a O. Isayev^{*b} and A. E. Roitberg^{*a}

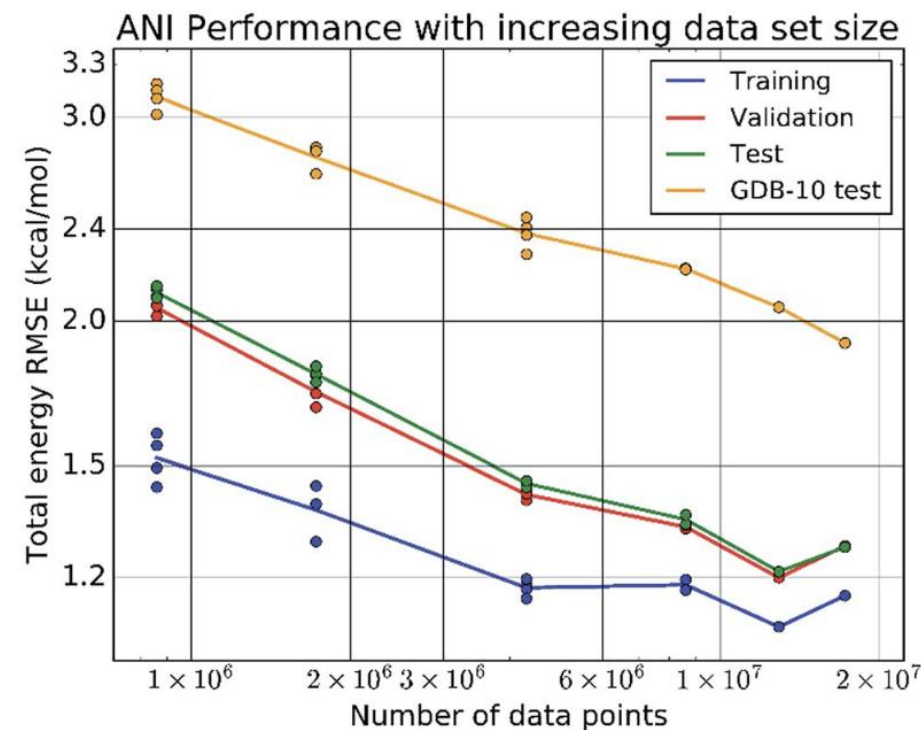


Fig. 3 Log-log plots of the training, validation, testing, and a random GDB-10 (molecules with 10 heavy atoms from the GDB-11 database) extensibility testing set of total energy errors vs. increasing number of data points in the training set. The sets of points converge to the final ANI-1 potential presented in this paper, trained on the full ANI-1 data set.

Interactive Python Time

`10_scaling_feature_engineering.ipynb`